

APOSTILA DIDÁTICA: ESTRUTURAS DE DADOS HOMOGÊNEAS EM LINGUAGEM C

Disciplina: Linguagem de Programação

Tópicos: Vetores (1D), Matrizes Multidimensionais e Manipulação de Strings

Material de Apoio Baseado em: Material 005 (Prof. Maromo)

1. Introdução às Estruturas Homogêneas (Arrays)

Em Linguagem C, variáveis primitivas simples (`int`, `float`, `char`, etc.) são capazes de armazenar apenas um único valor por vez. Quando uma aplicação demanda o armazenamento de múltiplos valores relacionados (como as notas de uma turma ou as coordenadas de uma malha), utiliza-se uma **matriz** (*array*).

Características Fundamentais:

1. **Homogeneidade:** Todos os elementos pertencem estritamente ao mesmo tipo de dado primitivo.
 2. **Alocação Contígua:** Os elementos são dispostos sequencialmente na memória RAM.
 3. **Acesso Direto via Índice:** O acesso a qualquer posição possui complexidade de tempo constante $O(1)$, realizado pelo cálculo do deslocamento de memória a partir do endereço base.
 4. **Indexação Base Zero:** Em C, o primeiro elemento reside no índice `0`, e o último no índice `tamanho - 1`.
-

2. Vetores Unidimensionais (1D)

2.1. Declaração e Sintaxe Geral

```
tipo_dado nome_vetor[tamanho];
```

Exemplos de Declaração:

```
int  numeros[5];      // Aloca espaço para 5 inteiros (5 * 4 bytes = 20 bytes)
float notas[10];      // Aloca espaço para 10 números em ponto flutuante
char  letras[26];     // Aloca espaço para 26 caracteres (26 * 1 byte = 26 bytes)
```

2.2. Organização e Endereçamento na Memória

Considere a declaração:

```
int v[5] = {10, 12, 14, 16, 18};
```

Assumindo que cada `int` ocupa 4 bytes e que o vetor inicia no endereço hipotético `0x1000`:

Índice (<i>i</i>)	Valor Armazenado (<i>v[i]</i>)	Expressão de Endereço	Endereço Real
0	10	$0x1000 + (0 * 4)$	0x1000
1	12	$0x1000 + (1 * 4)$	0x1004
2	14	$0x1000 + (2 * 4)$	0x1008
3	16	$0x1000 + (3 * 4)$	0x100C
4	18	$0x1000 + (4 * 4)$	0x1010

2.3. Carga Inicial (Inicialização)

Quando declarado localmente sem inicialização, o vetor contém lixo de memória. A carga inicial pode ser feita das seguintes formas:

```
// 1. Inicialização completa explícita
int v1[5] = {1, 2, 3, 4, 5};

// 2. Inicialização total com zeros
int v2[5] = {0}; // Todos os 5 elementos recebem 0

// 3. Inicialização parcial
int v3[5] = {10, 20}; // v3[0]=10, v3[1]=20, v3[2]=0, v3[3]=0, v3[4]=0

// 4. Dimensionamento automático pelo compilador
int v4[] = {2, 4, 6, 8, 10}; // Compilador define tamanho 5
```

Regra de Carga Parcial: Se forem fornecidos k valores para um vetor de tamanho N ($k < N$), as posições de índice k até $N - 1$ serão automaticamente zeradas pelo compilador.

2.4. Exemplo Prático: Leitura e Escrita de Vetor

```
#include <stdio.h>

#define TAM 5

int main() {
    int numeros[TAM];
    int i;
```

```
// Leitura dos elementos
printf("--- Entrada de Dados ---\n");
for (i = 0; i < TAM; i++) {
    printf("Informe o elemento [%d]: ", i);
    scanf("%d", &numeros[i]);
}

// Exibição dos elementos
printf("\n--- Elementos Armazenados ---\n");
for (i = 0; i < TAM; i++) {
    printf("numeros[%d] = %d\n", i, numeros[i]);
}

return 0;
}
```

3. Matrizes Bidimensionais (2D) e N-Dimensionais

Uma matriz bidimensional pode ser compreendida como uma tabela de dados estruturada em linhas e colunas (ou um "vetor de vetores").

3.1. Declaração e Inicialização 2D

```
tipo_dado nome_matriz[linhas][colunas];
```

Exemplos:

```
// Declaração sem inicialização
float tabela[3][4]; // 3 linhas e 4 colunas (12 elementos)

// Inicialização explícita por linhas
int mat[2][3] = {
    {1, 2, 3}, // Linha 0
    {4, 5, 6}  // Linha 1
};

// Inicialização com omissão da dimensão de linhas (colunas obrigatórias)
int matAuto[][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
}; // O compilador infere 3 linhas
```

3.2. Estrutura de Matrizes Tridimensionais (3D)

Representam cubos ou volumes de dados (ex.: camadas, linhas e colunas).

```
// Declaração de matriz 3D (2 blocos, 3 linhas, 4 colunas)
int cubo[2][3][4];

// Inicialização 3D (2x2x2)
int mat3D[2][2][2] = {
    {
        {1, 2},
        {3, 4}
    },
    {
        {5, 6},
        {7, 8}
    }
};
```

3.3. Exemplo Prático: Manipulação de Matriz 2D com Loops Aninhados

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define LIN 3
#define COL 3

int main() {
    int matriz[LIN][COL];
    int i, j;

    srand(time(NULL));

    // Preenchimento com valores aleatórios entre 1 e 9
    for (i = 0; i < LIN; i++) {
        for (j = 0; j < COL; j++) {
            matriz[i][j] = (rand() % 9) + 1;
        }
    }

    // Impressão tabular da matriz
    printf("Matriz %dx%d:\n", LIN, COL);
    for (i = 0; i < LIN; i++) {
        for (j = 0; j < COL; j++) {
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}
```

```
}  
  
return 0;  
}
```

4. Cadeias de Caracteres (Strings em C)

Em Linguagem C, não existe um tipo primitivo `string` nativo. Uma string é tratada como um **veter unidimensional de caracteres terminado pelo caractere nulo (`\0`)**.

4.1. Caractere vs. String

Aspecto	Caractere (<code>char</code>)	String (<code>char[]</code>)
Definição	Um único símbolo	Sequência de caracteres terminada em <code>\0</code>
Delimitador	Aspas simples: <code>'A'</code>	Aspas duplas: <code>"A"</code>
Consumo de Memória	1 byte	$N + 1$ bytes (onde 1 byte é reservado para <code>\0</code>)
Declaração	<code>char c = 'X';</code>	<code>char s[6] = "Hello";</code>

4.2. O Papel do Terminador `\0` (ASCII 0)

Se declaramos `char nome[100] = "ZE";`, as posições do array conterão:

```
Índice:  0    1    2    3    4    ...   99  
Valor:   ['Z', 'E', '\0', '?', '?', ..., '?]
```

As funções de manipulação de texto operam percorrendo os índices até encontrarem o byte `\0`.

4.3. Entrada e Saída de Strings

Funções de Saída:

- `printf("%s", str)`: Imprime a string até o caractere `\0`.
- `puts(str)`: Imprime a string e adiciona automaticamente uma quebra de linha (`\n`).

Funções de Entrada:

- `scanf("%s", str)`: Lê texto até o primeiro espaço em branco, tabulação ou quebra de linha (não lê frases completas).
- `gets(str)`: **Insegura e obsoleta desde o padrão C11**. Causa vulnerabilidade de estouro de buffer (*buffer overflow*) por não limitar a quantidade de caracteres lidos.

- `fgets(str, sizeof(str), stdin)`: **Forma padrão e segura**. Limita a leitura ao tamanho máximo do buffer, prevenindo transbordamentos.

```
#include <stdio.h>
#include <string.h>

int main() {
    char frase[100];

    printf("Digite uma frase: ");
    fgets(frase, sizeof(frase), stdin);

    // Opcional: remover o '\n' capturado pelo fgets
    frase[strcspn(frase, "\n")] = '\0';

    printf("Texto capturado: ");
    puts(frase);

    return 0;
}
```

5. Funções da Biblioteca `<string.h>` e Conversões (`<stdlib.h>`)

5.1. Manipulação Básica de Strings (`<string.h>`)

Função	Assinatura Resumida	Descrição / Operação
<code>strlen(s)</code>	<code>size_t strlen(const char *s)</code>	Retorna a quantidade de caracteres da string (excluindo o <code>\0</code>).
<code>strcpy(dest, orig)</code>	<code>char *strcpy(char *dest, const char *orig)</code>	Copia integralmente <code>orig</code> para <code>dest</code> .
<code>strncpy(dest, orig, n)</code>	<code>char *strncpy(char *dest, const char *orig, size_t n)</code>	Copia até <code>n</code> caracteres de <code>orig</code> para <code>dest</code> .
<code>strcat(dest, orig)</code>	<code>char *strcat(char *dest, const char *orig)</code>	Concatena <code>orig</code> ao final de <code>dest</code> .
<code>strncat(dest, orig, n)</code>	<code>char *strncat(char *dest, const char *orig, size_t n)</code>	Concatena até <code>n</code> caracteres de <code>orig</code> ao final de <code>dest</code> .

Exemplo de Manipulação:

```
#include <stdio.h>
#include <string.h>

int main() {
```

```
char origem[] = "Mundo";
char destino[50] = "Ola, ";

strcat(destino, origem);
printf("Resultado: %s\n", destino); // "Ola, Mundo"
printf("Comprimento: %zu caracteres\n", strlen(destino));

return 0;
}
```

5.2. Comparação de Strings (<string.h>)

As funções de comparação retornam:

- `0`: Strings idênticas.
- `< 0`: Primeira string é lexicograficamente menor que a segunda.
- `> 0`: Primeira string é lexicograficamente maior que a segunda.

Função	Descrição
<code>strcmp(s1, s2)</code>	Compara <code>s1</code> e <code>s2</code> considerando maiúsculas e minúsculas (<i>case-sensitive</i>).
<code>strncmp(s1, s2, n)</code>	Compara os primeiros <code>n</code> caracteres de <code>s1</code> e <code>s2</code> .
<code>strcasecmp(s1, s2)</code>	Compara <code>s1</code> e <code>s2</code> ignorando maiúsculas/minúsculas (<i>POSIX/GNU</i>).

5.3. Formatação e Conversão de Tipos (<stdio.h> e <stdlib.h>)

- `sprintf(buffer, formato, ...)`: Formata dados e grava o resultado em uma string.
- `sscanf(buffer, formato, ...)`: Lê dados formatados a partir de uma string em vez do teclado.
- `atoi(str)`: Converte string numérica para inteiro (`int`).
- `atol(str)`: Converte string numérica para inteiro longo (`long int`).
- `atof(str)`: Converte string numérica para ponto flutuante (`double`).

Exemplo de Formatação e Conversão:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char buffer[100];
    char dados[] = "100 45.75";
    int n1;
    double n2;

    // Leitura a partir de string com sscanf
    sscanf(dados, "%d %lf", &n1, &n2);
}
```

```
// Formatação de string com sprintf
sprintf(buffer, "Valores lidos: Inteiro = %d | Ponto Flutuante = %.2f", n1,
n2);
puts(buffer);

// Conversão direta
char strNumero[] = "250";
int valor = atoi(strNumero);
printf("Valor convertido somado a 10: %d\n", valor + 10);

return 0;
}
```

6. Lista de Exercícios Práticos

Módulo 1: Vetores Unidimensionais

1. **Soma dos Elementos:** Escreva um programa em C que leia um vetor de 10 números inteiros, calcule e imprima a soma de todos os seus elementos.
2. **Maior e Menor Valor:** Desenvolva um programa que leia um vetor de 8 posições e determine o maior valor e o menor valor, informando suas respectivas posições (índices).
3. **Inversão de Vetor:** Crie um programa que leia um vetor de 15 números inteiros e apresente o vetor em ordem inversa à leitura.
4. **Contagem de Pares e Ímpares:** Faça um programa que leia um vetor de 20 números inteiros e exiba a quantidade de valores pares e a quantidade de valores ímpares.
5. **Substituição de Negativos:** Escreva um programa que leia um vetor de 12 posições. Substitua todos os números negativos presentes no vetor por zero e imprima o vetor resultante.

Módulo 2: Matrizes Bidimensionais

1. **Soma de Matrizes:** Escreva um programa que leia duas matrizes 3×3 de inteiros e gere uma terceira matriz contendo a soma dos elementos correspondentes.
2. **Diagonal Principal:** Desenvolva um programa que leia uma matriz quadrada 4×4 e exiba apenas os valores situados na diagonal principal ($i = j$).
3. **Matriz Transposta:** Crie um programa que leia uma matriz 3×3 e gere e exiba a sua transposta (troca de linhas por colunas: $M^T[j][i] = M[i][j]$).
4. **Maior Valor e Coordenadas:** Elabore um programa que leia uma matriz 5×5 , localize o maior valor armazenado e imprima esse valor juntamente com a linha e coluna onde ele se encontra.
5. **Multiplicação por Escalar:** Escreva um programa que leia uma matriz 2×2 e um número inteiro multiplicador (escalar). Multiplique cada elemento da matriz por esse escalar e imprima o resultado.

Módulo 3: Manipulação de Strings

1. **Concatenação Segura:** Escreva um programa que receba duas strings do usuário e, utilizando a função `strcat()` (ou `strncat()`), concatene a segunda ao final da primeira, imprimindo o resultado.

2. **Comparação de Textos:** Desenvolva um programa que leia duas palavras e verifique se são exatamente iguais usando `strcmp()`.
 3. **Cópia com Limite de Caracteres:** Crie um programa que leia uma frase e um número inteiro n . Copie os primeiros n caracteres para uma nova string via `strncpy()`, garantindo a inserção do `\0` final, e exiba a nova string.
 4. **Conversão e Cálculo Numérico:** Elabore um programa que leia duas strings: uma representando um valor inteiro e outra um valor decimal. Converta-as usando `atoi()` e `atof()` e exiba a soma numérica dos dois valores.
 5. **Formatação e Extração:** Escreva um programa que leia nome, idade e altura. Formate esses dados em uma única string com `sprintf()`. Em seguida, usando `sscanf()`, extraia novamente os dados para variáveis individuais e confirme os valores na tela.
-

7. Referências Bibliográficas

- DAMAS, Luís. **Linguagem C**. Rio de Janeiro: LTC, 2007.
- SCHILDT, Herbert. **C Completo e Total**. 3. ed. São Paulo: Pearson / Makron Books, 1997.
- KERNIGHAN, Brian W.; RITCHIE, Dennis M. **C: A Linguagem de Programação Padrão ANSI**. Rio de Janeiro: Campus, 1989.